P1502R1
Standard library header units for C++20

Published Proposal, 2019-07-18

This version:
http://wg21.link/p1502r1

Author:
Richard Smith (Google)

Audience:
LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes minimal support for modular use of the C++ standard library via header units.

Table of Contents

§ 1. Background

With the anticipated inclusion of modules in the C++20 langauge, we need a natural and portable mechanism to import the standard library into a modules-enabled compilation.

This provides us with a unique opportunity: when converting the standard library into a set of named modules (universally assumed to be given names beginning `std`), we can reconsider which pieces of the library belong together and which can be split apart, with few backwards compatibility constraints. (There are techniques that permit standard library vendors to maintain ABI across such a reorganization, and we trust the them to use those techniques as they see fit.)

Indeed, various parties are investigating such reorganizations of the standard (see [P0581R1], [P1212R0], [P1453R0]), and we anticipate such investigation will lead to improvements in the overall structure of the standard library as exposed to modern modular C++. If such a solution is ready in time for C++20, we would welcome such a development.

However, it is not clear that such a solution will be ready in time, and the worst outcome is looming: that different implementations might expose the C++ standard library in different ways, fracturing the language and ecosystem. While modules gives us an opportunity to improve upon the status quo in this area, we can decouple such improvements from the addition of the modules language feature and basic support for the standard library therein, and doing so is good engineering practice.

§ 2. Goals

We seek to solve three problems:

1) Provide a minimal, uninventive mechanism to expose the existing standard library organization to modular compilations.

2) Do not get in the way of, or create any impediment to, a proper reorganization of the standard library into named modules.

3) Reserve space for such a future reorgnization.

§ 3. Approach

§ 3.1. Standard library header units

We propose to guarantee that the C++ standard library headers can be imported as header units:

```
// C++17 or C++20
#include <vector>
#include <tuple>
```

```
// C++20
import <vector>;
import <tuple>;
```

This has a number of consequences, such as:

- No reorganization of any library contents nor ABI changes
- Existing `#include`s of standard library headers transparently turn into module imports in C++20
- Additional names can "leak" out of standard library header units, just as they can "leak" out of standard library headers today (for example, it's unspecified whether `<vector>` includes `<utility>` today)
- Some situations that were formerly undefined behavior now become ill-formed or well-defined:
    - Defining a keyword or library identifier then including or importing a library header need not be undefined, as the `#define` does not affect the standard library header unit.
    - Including a standard library header inside a namespace becomes ill-formed rather than undefined behavior.

We propose that only the C++ standard library headers (excluding both the `<cfoo>` headers and the deprecated `<foo.h>` headers) are affected. The `<foo.h>` headers are included from shared C / C++ header files, frequently in `extern "C"` blocks, for which `#include` translation would create trouble. In addition, the underlying C standard library sometimes gives additional semantics to macros defined before the `<foo.h>` headers are included (for example `__need_NULL`, `_POSIX_SOURCE`), and the behavior of the `<cassert>` header differs based on which macros are defined beforehand. Per 16.5.2.2 [using.headers]/2:

> A translation unit may include library headers in any order (Clause 5). Each may be included more than once, with no effect different from being included exactly once, except that the effect of including either `<cassert>` or `<assert.h>` depends each time on the lexically current definition of `NDEBUG`.

§ 3.2. `std` module land-grab

We propose to reserve modules whose names begin with a `std` name component or `std` followed by a sequence of digits.

If any module name component is a reserved identifier, the module name is reserved for implementations (eg, `std._Vendor.something`); all other names are reserved for future standardization (eg, `std.vector`).

§ 4. Wording

§ 4.1. §10.1 Module units and purviews [module.unit]

Change in 10.1 [module.unit] paragraph 1:

> A *module unit* is a translation unit that contains a *module-declaration*. A *named module* is the collection of module units with the same *module-name*. The *identifier*s `module` and `import` shall not appear as *identifier*s in a *module-name* or *module-partition*. All *module-name*s beginning with an *identifier* consisting of `std` followed by zero or more *digit*s or containing a reserved *identifier* (5.10 [lex.name]) are reserved and shall not be specified in a *module-declaration*; no diagnostic is required. If any *identifier* in a reserved *module-name* is a reserved identifier, the module name is reserved for use by C++ implementations; otherwise it is reserved for future standardization. The optional *attribute-specifier-seq* appertains to the *module-declaration*.

§ 4.2. §10.3 Import declaration [module.import]

Change in 10.3 [module.import] paragraph 3:

> [...] An *importable header* is a member of an implementation-defined set of headers that includes all importable C++ library headers (16.5.1.2 [headers]) . [...]

§ 4.3. §16.5.1.2 Headers [headers]

Add a new paragraph after 16.5.1.2 [headers] paragraph 3:

> The headers listed in Table 19 [C++ library headers], or, for a freestanding implementation, the subset of such headers that are provided by the implementation, are collectively known as the *importable C++ library headers*. [*Note*: Importable C++ library headers can be imported as module units (10.3 [module.import]). — *end note*] [*Example*:
>
> ```
> import <vector>;          // imports the <vector> header unit
> std::vector<int> vi;      // OK
> ```
>
> — *end example*]

§ 4.4. §16.5.2.2 Headers [using.headers]

Change in 16.5.2.2 [using.headers] paragraph 1:

> The entities in the C++ standard library are defined in headers, whose contents are made available to a translation unit when it contains the appropriate `#include` preprocessing directive (15.2) or the appropriate `import` declaration (10.3 [module.import]).

Change in 16.5.2.2 [using.headers] paragraph 3:

> A translation unit shall include a header only outside of any declaration or definition and, in the case of a module unit, only in its *global-module-fragment* , and shall include the header or import the corresponding header unit lexically before the first reference in that translation unit to any of the entities declared in that header. No diagnostic is required.

§ 4.5. Feature test macro

No feature test macro is proposed. Code that wants to work in the absence of this feature can use `#include` instead of `import`.

§ References

§ Informative References

[P0581R1]

Marshall Clow, Beman Dawes, Gabriel Dos Reis, Stephan T. Lavavej, Billy O'Neal, Bjarne Stroustrup, Jonathan Wakely. Standard Library Modules. 11 February 2018. URL: https://wg21.link/p0581r1

[P1212R0]

Ben Craig. Modules and Freestanding. 6 October 2018. URL: https://wg21.link/p1212r0

[P1453R0]

Bryce Adelstein Lelbach. Modularizing the Standard Library is a Reorganization Opportunity. 21 January 2019. URL: https://wg21.link/p1453r0